Cooperative Robotics

Authors: “MILAD RABIEI, ARIAN TAVOUSI”
Email: “s7215122@studenti.unige.it , s7131598@studenti.unige.it”
Date: “06/01/2026”

General notes

- Exercise 1 is done with the ROBUST matlab main and unity visualization tools. Exercises 2-3 are done with the Franka simulation tools.
- Comment and discuss the simulations, in a concise scientific manner. Further comments, other than the questions, can be added, although there is no need to write 10 pages for each exercise.
- Aid the discussion with screenshots of the simulated environment (compress them to maintain a small overall file size), and graphs of the relevant variables (i.e. activation functions of inequality tasks, task variables, and so on). Graphs should always report the units of measure in both axes, and legends whenever relevant.
- You can add vertical bars to plots to highlight periods of time (e.g. when one action is active), or vertical ones to highlight thresholds (e.g. the minimum altitude value).
- Report the thresholds whenever relevant.
- Report the mathematical formula employed to derive the task jacobians and the control laws when asked, including where they are projected.
- If needed, part of the code can be inserted as a discussion reference.
- Provide your answers in the subsection where the question is made.

Use the following template when you need to discuss the hierarchy of tasks of a given action or set of actions. Please report the tasks in the same order as in the unified hierarchy implemented in your code.

Table 1: Example of actions/hierarchy table: a number in a given cell represents the priority of the control task (row) in the hierarchy of the control action (column). The type column indicates whether the objective is an equality (E) or inequality (I) one.

Task	Type	$\mathcal{A}_1$	$\mathcal{A}_2$	$\mathcal{A}_3$
Task name A	I	1		1
Task name B	I	2	1	
Task name C	E		2	2

1 Exercise 1: Implement a Complete Mission

Implement several actions to reach a point, land, and then perform the manipulation of a target object.

Initialize the vehicle in a position that allows you to test all the tasks. Then, use a "safe waypoint navigation action" to reach the following position:

$$\begin{bmatrix} 10.5 & 37.5 & -38 & 0 & -0.06 & 0.5 \end{bmatrix}^\top$$

Then land, aligning to the nodule. In particular, the x axis of the vehicle should align to the projection, on the inertial horizontal plane, of the unit vector joining the vehicle frame to the nodule frame. Once landed, implement a "fixed-based manipulation action" to reach the target nodule (mimicking the scanning of the nodule). During this manipulation phase, the vehicle should not move for any reason. Considering that the goal position for the vehicle could be not so precise with respect to the position of the nodule, implement a solution that guarantees that, once landed, the nodule is certainly within the manipulator's workspace. If needed, you can change also the vehicle's goal position if you need to stress the system and force certain tasks to activate (e.g., the distance to the nodule one).

1.1 Q1: Report the unified hierarchy of tasks used, the actions and their tasks, specifying their priorities in each action.

Control architecture overview (tasks, actions, and task priority)

The mission is implemented as a sequence of *actions*. Each action corresponds to a *stack of tasks* solved with a task-priority inverse-kinematics scheme (iCAT). The control vector is the UVMS generalized velocity:

$$\dot{\mathbf{y}} = \begin{bmatrix} \dot{\mathbf{q}} \\ \boldsymbol{\nu} \end{bmatrix} \in \mathbb{R}^{13}, \qquad \dot{\mathbf{q}} \in \mathbb{R}^7, \quad \boldsymbol{\nu} = \begin{bmatrix} \mathbf{v} \\ \boldsymbol{\omega} \end{bmatrix} \in \mathbb{R}^6, \tag{1}$$

where $\dot{\mathbf{q}}$ are the arm joint velocities and $\boldsymbol{\nu}$ is the vehicle velocity expressed in the vehicle frame (3 linear + 3 angular components). Each task i provides:

- a reference task velocity $\dot{\bar{\mathbf{x}}}_i$,
- a task Jacobian $\mathbf{J}_i$ mapping $\dot{\mathbf{y}} \mapsto \dot{\mathbf{x}}_i$,
- an activation (scalar or diagonal) $\mathbf{A}_i$ that smoothly enables/disables the task.

The iCAT module enforces strict priorities: tasks earlier in the stack have higher priority. The mission requires switching between actions (navigation → landing/alignment → fixed-base manipulation). To avoid discontinuities, the code uses an *action transition* mechanism that merges the activations of consecutive tasks (of actions) for a fadeing-style transition.

Mission actions and task sets

The complete mission is organized into three main actions:

1. **Action 1 – Safe waypoint navigation:** drive the vehicle to the waypoint while guaranteeing safety (altitude floor and stable attitude) and keeping the manipulator in a benign configuration.
2. **Action 2 – Landing with alignment to the nodule:** descend until contact/near-contact, while aligning the vehicle yaw toward the nodule direction projected on the horizontal plane.
3. **Action 3 – Fixed-base manipulation (scan):** lock the vehicle pose (no motion allowed), and use the arm tool task to reach/scan the nodule.

Task definitions used in the implementation

All tasks have been placed inside the tasks folder in the codebase. Below is the list of tasks and their roles in the mission:

- **Vehicle position task** ($\mathcal{T}_{VP}$): drives the vehicle translation and orientation to a desired waypoint represented in world frame. In the code it can be called by `TaskVehiclePositionA`.

- **Attitude/leveling task** ($\mathcal{T}_{ATT}$): keeps roll/pitch bounded (aligns steady horizontal plane). In the code it can be called with `TaskAttitudeAlignment`.
- **Altitude safety task** ($\mathcal{T}_{ALT}$): may activate when altitude becomes too small and enforces a minimum altitude (constraint via activation). In the code this is `TaskAltitudeControl` with a bell-shaped activation.
- **Landing task** ($\mathcal{T}_L$): commands a controlled descent. In the code this is `TaskLand`.
- **Alignment to target task** ($\mathcal{T}_{AtD}$): aligns the vehicle x axis with the projected direction from the vehicle to the nodule on the inertial horizontal plane.
- **Tool pose task** ($\mathcal{T}_{TP}$): drives the tool/end-effector to the nodule pose (for scanning). In the code this is `TaskToolPositionA` (absolute target pose). Noted that this task uses **vehicle-arm coordination scheme**, so manipulator joint commands are generated knowing the vehicle's motion.

Table 2: Unified hierarchy of tasks for the implemented actions

Task	Type	$\mathcal{A}_1$	$\mathcal{A}_2$	$\mathcal{A}_3$
Altitude control (Alt)	I	1		
Vehicle position (VP)	E	2		
Attitude heading (AtD)	E		1	
Attitude leveling (Att)	I		2	
Landing (L)	I		3	
Tool position (TP)	E			1

Safety tasks should be the highest priority tasks. The altitude safety task is kept at the highest priority because it acts as a safety "floor" and should override other objectives if needed. During landing, the alignment tasks are placed above the descent (objective) task so that the vehicle achieves the correct heading while descending. During fixed-base manipulation, the vehicle-arm coordination outputs a modified Jacobian and Reference vector (focused on the arm) that encodes the compensation, ensuring the vehicle does not make additional movements while the arm executes the scan.

Action switching and unified stack construction (code-level)

The transition logic is implemented by merging the task ID lists of two consecutive actions and assigning each merged level a *type*:

- type 0: constant (present in both actions),
- type -1: decreasing (fade out),
- type 1: increasing (fade in).

implemented in function `generatePriorityList.m`. During transitions, each task activation is multiplied by a secondary priority α:

$$\mathbf{A}_i(t) \leftarrow \alpha_i(t)\,\mathbf{A}_i, \qquad \alpha_i(t) = \begin{cases} \mathrm{IncBell}(0, t_{\max}; t) & \text{if type } = 1, \\ \mathrm{DecBell}(0, t_{\max}; t) & \text{if type } = -1, \\ 1 & \text{if type } = 0, \end{cases} \tag{2}$$

with smooth bell-shaped functions. This prevents discontinuities in commanded velocities and helps reduce jerk.

1.2 Q2: Comment the behaviour of the robot, supported by relevant plots. For example, show the altitude and the altitude activation to show that when the altitude becomes too small, the tasks gets activated and a minimum value is guaranteed.

The complete mission can be read as three phases, each dominated by different high-priority tasks:

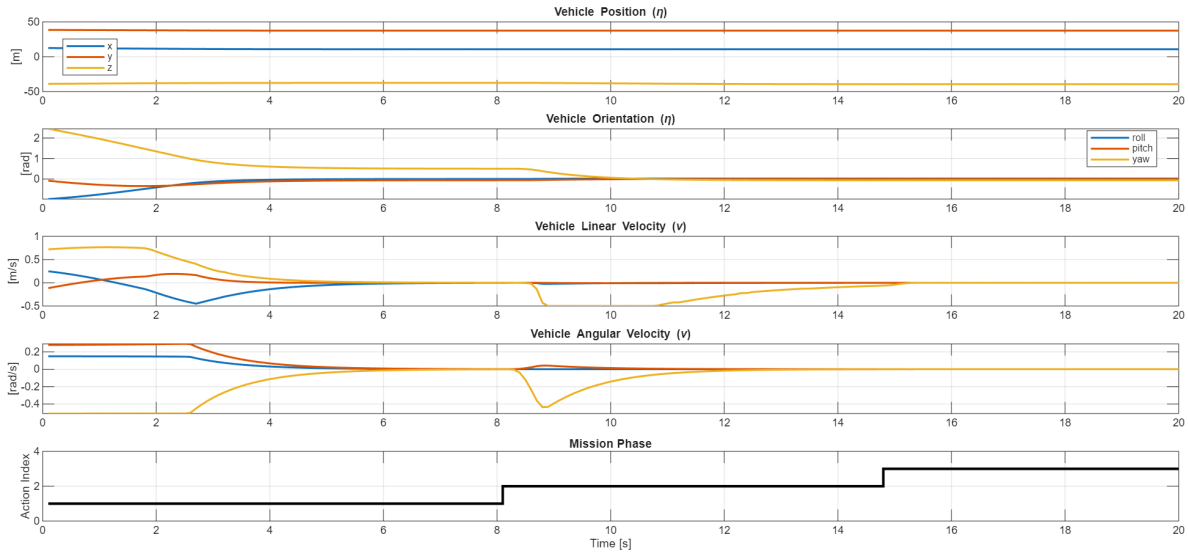


Figure 1: Vehicle state evolution during the mission

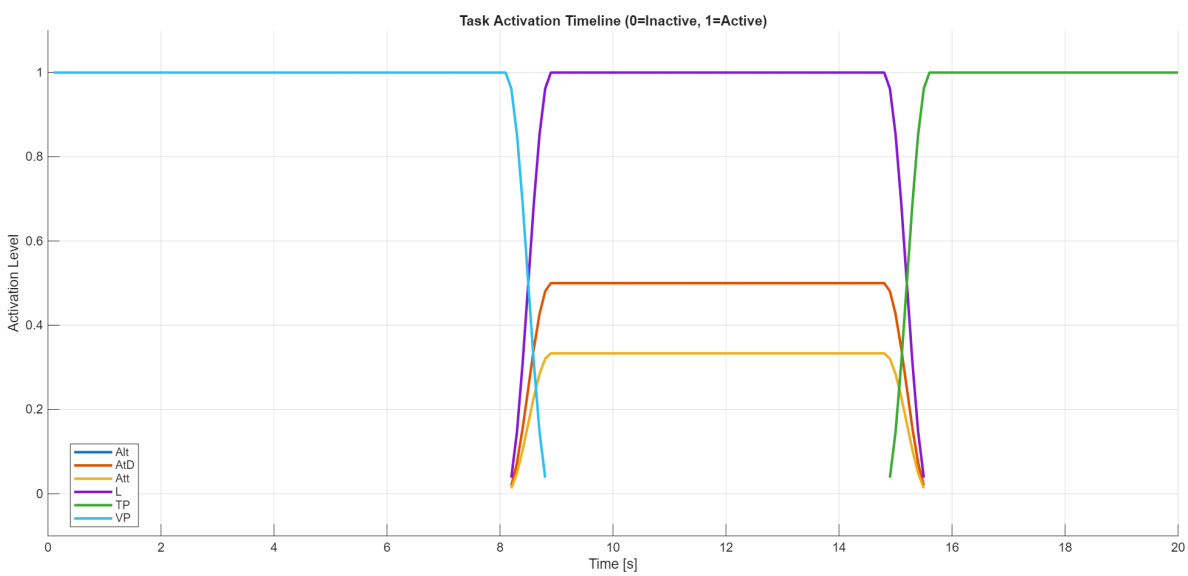


Figure 2: Task activation timeline across the mission actions

Phase 1 (Safe waypoint navigation). At the beginning, the vehicle starts from an initial pose (e.g., roll/pitch not exactly zero and altitude close enough to trigger altitude-control). The vehicle position task $\mathcal{T}_{VP}$ drives the vehicle toward the waypoint $\begin{bmatrix}10.5, & 37.5, & -38\end{bmatrix}^\top$ while $\mathcal{T}_{ALT}$ ensures the vehicle never violates the minimum altitude, and $\mathcal{T}_{ATT}$ prevents excessive roll/pitch excursions while the vehicle translates.

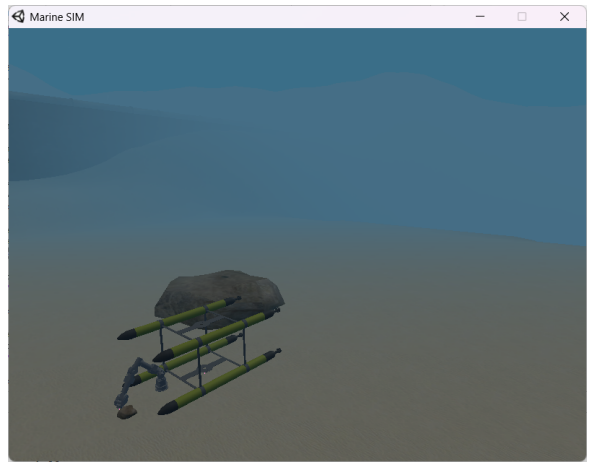
In plots it is clear that as the altitude task is active, the vehicle's altitude strives not to underpass a minimum, and as it reaches that minimum the activation wears off. Also, at first the roll and pitch

are way off, and so the task optimizes them to zero and then the activation fades off as the inequality gets satisfied.

Phase 2 (Landing with alignment). Once the waypoint is reached, position error below a threshold, and as action 1's activations wear off and action 2 becomes more transitionally active, it switches to landing phase. In this phase the landing task $\mathcal{T}_L$ commands a descent, the alignment task $\mathcal{T}_{AtD}$ directs the vehicle x axis towards the nodule (after projection on the horizontal plane), the vehicle XY hold keeps the horizontal coordinates stable while descending, not to crash from other angles.

In plots, altitude decreases smoothly, and the task activation timeline shows alignment and landing turning on. In the final figure for the robot it is clear the heading of the robot is towards the nodule.

(a) Before touchdown: vehicle approaching/aligning toward the nodule.



(b) After touchdown: fixed-base manipulation/scan configuration near the nodule.

Figure 3: Unity simulation snapshots of the mission: (a) landing/alignment phase, (b) manipulation/scan phase.

Phase 3 (Fixed-base manipulation / scan). After touchdown, the system switches to the fixed-base manipulation action. The key observable effect is vehicle velocities keep at their previous values (vehicle-arm coordination scheme), the tool task becomes active and drives the tool toward the nodule pose, joint velocities become nonzero while the vehicle remains in previous motion.

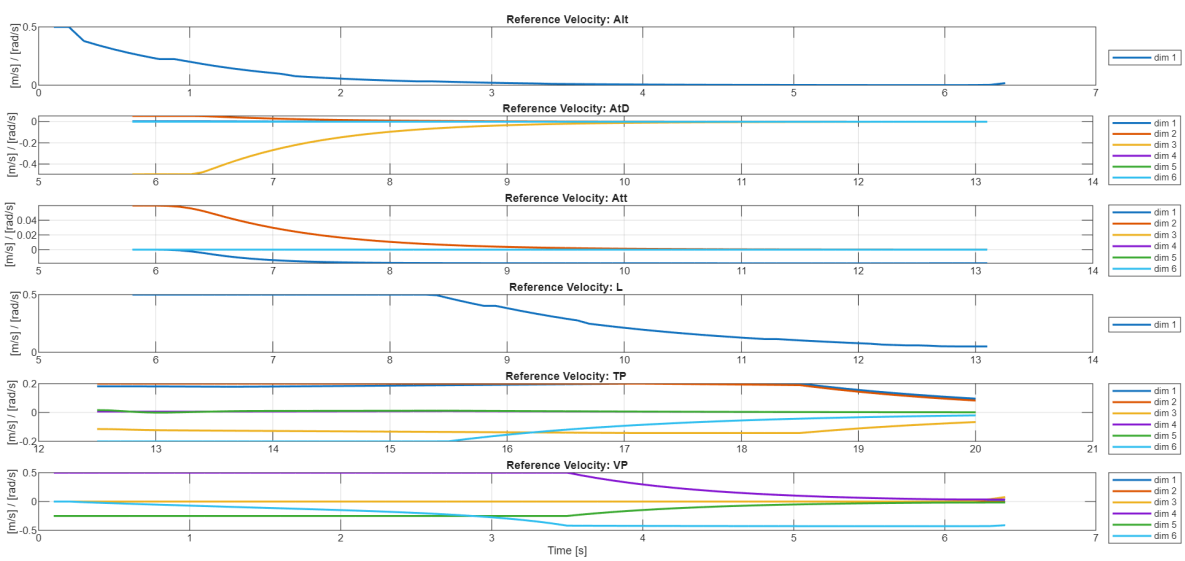


Figure 4: Task reference velocities for each task during the mission (Alt, AtD, Att, L, TP, VP).

Note about activation of inequality tasks

The altitude safety task uses a smooth activation that increases when the measured altitude approaches a critical range.

$$a_{ALT}(h) = \begin{cases} 1 & h \leq h_{\min}, \\ \frac{1}{2}\left(1 + \cos\left(\pi \frac{h - h_{\min}}{h_{\max} - h_{\min}}\right)\right) & h_{\min} < h < h_{\max}, \\ 0 & h \geq h_{\max}, \end{cases} \tag{3}$$

where h is altitude, $h_{\min}$ is the hard floor, and $h_{\max}$ is the altitude above which the safety task is completely inactive. `Decreasing/Increasing BellShapedFunction(h_min, h_max, 0, 1, altitude)` implement the activation generation. When the vehicle is far from the ground ($h \geq h_{\max}$), $a_{ALT} \approx 0$ so the task does not affect the motion. when the vehicle gets too close to the ground, $a_{ALT} \to 1$ and the solver prioritizes increasing altitude (or reducing descent), and therefore preventing violation of $h_{\min}$.

Transition functionality in the Action Manager

The transition functionality implemented in `ActionManager_TAVC` guarantees smooth switching between consecutive actions by avoiding discontinuities in the commanded velocities. When an action switch is requested, the task stacks of the current and next actions are merged into a unified hierarchy. Each task is classified as: (i) constant (present in both actions), (ii) fading out (present only in the previous action), or (iii) fading in (present only in the next action).

For fading tasks, the original activation matrix is multiplied by a time-dependent bell-shaped scaling factor. This progressively reduces the influence of outgoing tasks and increases the influence of incoming ones, while tasks common to both actions remain fully active. This mechanism ensures continuity of task activations and prevents impulsive joint or vehicle velocity commands during action transitions.

Listing 1: Action transition handling in `ActionManager_TAVC`

```
% Build unified task list between current and next action
mergedTaskIDs = union(currentTaskIDs, nextTaskIDs);

for i = 1:length(mergedTaskIDs)
    if ismember(mergedTaskIDs{i}, currentTaskIDs) && ...
       ismember(mergedTaskIDs{i}, nextTaskIDs)
        taskType(i) = 0; % constant task
    elseif ismember(mergedTaskIDs{i}, currentTaskIDs)
        taskType(i) = -1; % fade-out task
    else
        taskType(i) = 1; % fade-in task
    end
end

% Scale task activations during transition
for i = 1:length(taskType)
    if taskType(i) == 1
        A{i} = alpha_inc(t) * A{i}; % fade-in
    elseif taskType(i) == -1
        A{i} = alpha_dec(t) * A{i}; % fade-out
    end
end
```

In order to complete it, this functionality for actual imposition of transition has been implemented:

```
if obj.inTransition
dt = currentTime - obj.t_switch;
if dt >= obj.t_max
    obj.inTransition = false;
    obj.activeStack = obj.actions{obj.currentAction};
```

```
        obj.transitionPlan = struct('type', num2cell(zeros(1, length(obj.activeStack))));
        fprintf('[ActionManager] Transition Complete at t=%.2f\n', currentTime);
    end
end
```

1.3 Q3: What is the Jacobian relationship for the Alignment to Target control task? How was the task reference computed?

Task objective

The objective is to align the vehicle x-axis to the horizontal direction from the vehicle to the nodule. Let

- $\mathbf{p}_v \in \mathbb{R}^3$ be the vehicle position in the world frame (in code: `robot.eta(1:3)`),
- $\mathbf{p}_n \in \mathbb{R}^3$ be the nodule position in the world frame (stored in `obj.nodule_position`),
- $\mathbf{R}_{wv} \in SO(3)$ be the vehicle rotation from vehicle to world (in code: `wRv = robot.wTv(1:3,1:3)`),
- $\mathbf{e}_1 = [1\ 0\ 0]^\top$ be the vehicle x axis expressed in vehicle coordinates.

The direction vehicle $\to$ nodule is

$$\mathbf{d} = \mathbf{p}_n - \mathbf{p}_v. \tag{4}$$

To enforce *horizontal* alignment, the implementation projects onto the inertial horizontal plane by setting the vertical component to zero:

$$\mathbf{d}_\perp = \begin{bmatrix} d_x \\ d_y \\ 0 \end{bmatrix}, \qquad \hat{\mathbf{d}}_\perp = \frac{\mathbf{d}_\perp}{\|\mathbf{d}_\perp\|}. \tag{5}$$

The current vehicle x axis expressed in the world frame is:

$$\mathbf{x}_v = \mathbf{R}_{wv}\,\mathbf{e}_1. \tag{6}$$

The alignment condition is

$$\mathbf{x}_v \parallel \hat{\mathbf{d}}_\perp. \tag{7}$$

Task reference generation (reference twist)

The implemented task is a *6D twist task* whose reference commands only an angular velocity (no translation):

$$\dot{\bar{\mathbf{x}}}_{AtD} = \begin{bmatrix} \boldsymbol{\omega}_{ref} \\ \mathbf{v}_{ref} \end{bmatrix} \in \mathbb{R}^6, \qquad \mathbf{v}_{ref} = \mathbf{0}_{3\times 1}. \tag{8}$$

The angular error is constructed using the cross product:

$$\mathbf{e}_\omega = \mathbf{x}_v \times \hat{\mathbf{d}}_\perp. \tag{9}$$

Based on Versor Lemma if θ is the angle between $\mathbf{x}_v$ and $\hat{\mathbf{d}}_\perp$, then

$$\|\mathbf{e}_\omega\| = \|\mathbf{x}_v\|\,\|\hat{\mathbf{d}}_\perp\| \sin\theta = \sin\theta, \tag{10}$$

and $\mathbf{e}_\omega$ points along the rotation axis that would rotate $\mathbf{x}_v$ toward $\hat{\mathbf{d}}_\perp$ (right-hand rule). A proportional law generates the desired angular velocity:

$$\boldsymbol{\omega}_{ref} = k_p\,\mathbf{e}_\omega, \qquad k_p > 0 \tag{11}$$

.

Degenerate case and Saturation. If $\|\mathbf{d}_\perp\| \approx 0$ (vehicle already at the nodule in horizontal projection), the direction is undefined. The implementation handles this by setting the reference to zero:

$$\|\mathbf{d}_\perp\| < \varepsilon \;\Rightarrow\; \dot{\bar{\mathbf{x}}}_{AtD} = \mathbf{0}_{6\times 1}. \tag{12}$$

and similar in most tasks to avoid excessively aggressive rotations, the reference angular part is saturated.

Jacobian relationship

The solver uses the UVMS generalized velocity

$$\dot{\mathbf{y}} = \begin{bmatrix} \dot{\mathbf{q}} \\ \boldsymbol{\nu} \end{bmatrix} = \begin{bmatrix} \dot{\mathbf{q}} \\ \mathbf{v}_b \\ \boldsymbol{\omega}_b \end{bmatrix} \in \mathbb{R}^{13}, \tag{13}$$

where $\dot{\mathbf{q}} \in \mathbb{R}^7$ are arm joint velocities, and $\boldsymbol{\nu} \in \mathbb{R}^6$ is the vehicle twist expressed in the vehicle/body frame (linear $\mathbf{v}_b \in \mathbb{R}^3$ and angular $\boldsymbol{\omega}_b \in \mathbb{R}^3$). The task output is a world-frame twist

$$\dot{\mathbf{x}}_{AtD} = \begin{bmatrix} \boldsymbol{\omega}_w \\ \mathbf{v}_w \end{bmatrix}. \tag{14}$$

With the standard rotation mapping between body and world angular velocities,

$$\boldsymbol{\omega}_w = \mathbf{R}_v^w \, \boldsymbol{\omega}_b, \tag{15}$$

and since this task does not depend on arm joints nor vehicle linear velocity, the Jacobian used is:

$$\dot{\mathbf{x}}_{AtD} = \mathbf{J}_{AtD}\, \dot{\mathbf{y}}, \qquad \mathbf{J}_{AtD}(\mathbf{R}_{wv}) = \begin{bmatrix} \mathbf{0}_{3\times 7} & \mathbf{0}_{3\times 3} & \mathbf{R}_{wv} \\ \mathbf{0}_{3\times 7} & \mathbf{0}_{3\times 3} & \mathbf{0}_{3\times 3} \end{bmatrix} \in \mathbb{R}^{6\times 13}. \tag{16}$$

This matches `obj.J = [zeros(3,7), zeros(3,3), wRv; zeros(3,13)]`.

Activation matrix

Although the task is expressed as a 6D twist, only the angular part is meant to be enforced. This is implemented through a diagonal activation matrix:

$$\mathbf{A}_{AtD} = \mathrm{diag}\big(1, 1, 1, 0, 0, 0\big), \tag{17}$$

so that the solver penalizes/tracks the angular components while completely ignoring the linear part.

$$\mathbf{A}_{AtD}\Big(\dot{\bar{\mathbf{x}}}_{AtD} - \mathbf{J}_{AtD}\dot{\mathbf{y}}\Big) \to \mathbf{0}, \tag{18}$$

.

Implementation snippet

```
function obj = TaskAttitudeDirect(nodule_pos)
    obj.nodule_position = nodule_pos(:);
    obj.kp = 1.0;
    obj.task_name = "AtDX";
end

function updateReference(obj, robot)
    p_v = robot.eta(1:3);
    d = obj.nodule_position - p_v;
    d(3) = 0;
    if norm(d) < 1e-6
        obj.xdotbar = zeros(6,1);
        return;
```

```
    end
    d_hat = d / norm(d);
    wRv = robot.wTv(1:3,1:3);
    x_v = wRv * [1; 0; 0];
    e_ang = cross(x_v, d_hat);
    v_ang = obj.kp * e_ang;
    v_lin = zeros(3,1);

    obj.xdotbar = [v_ang; v_lin];
    limit = 0.5;
    if norm(obj.xdotbar(1:3)) > limit
        obj.xdotbar(1:3) = obj.xdotbar(1:3) ...
            * limit / norm(obj.xdotbar(1:3));
    end
end

function updateJacobian(obj, robot)
    wRv = robot.wTv(1:3,1:3);

    obj.J = [zeros(3,7), zeros(3,3), wRv;
             zeros(3,13)];
end

function updateActivation(obj, ~)
    obj.A = diag([1 1 1 0 0 0]);
end
```

1.4 Q4: What have you done to guarantee that the nodule is within the manipulator's workspace?

In our implementation

In our specific simulation run, we did not implement an explicit workspace-guarantee mechanism. The selected vehicle goal and the initial configuration were selected to place the nodule inside the arm workspace after landing, so the fixed-base manipulation succeeded without requiring additional logic.

However, relying on "getting lucky" is fragile. For this reason, below we describe a strategy that may guarantee the nodule lies within the reachable region of the manipulator at touchdown.

Problem statement and failure mode

Even if the vehicle reaches the planned goal position, the nodule may still be outside the manipulator workspace.

If the system switches to fixed-base manipulation (as it does since the vehicle is landed and cannot move) while the nodule is unreachable, the tool task error cannot converge, the arm may saturate at joint limits, and the behaviour appears "stuck".

Solution: Inequality task for a reachable sphere around the nodule

Let $\mathbf{p}_v \in \mathbb{R}^3$ be the vehicle position in the world frame and $\mathbf{R}_v^w \in SO(3)$ the vehicle orientation. Let $\mathbf{p}_{vb}$ be the fixed translation from the vehicle frame to the manipulator base frame. Then the manipulator base position in world coordinates is

$$\mathbf{p}_b = \mathbf{p}_v + \mathbf{R}_v^w \, \mathbf{p}_{vb}. \tag{19}$$

Let $\mathbf{p}_n$ be the nodule position (world frame). A conservative reachability constraint is:

$$d(t) \;=\; \|\mathbf{p}_n - \mathbf{p}_b\| \;\leq\; r_{\text{safe}}, \tag{20}$$

where $r_{\text{safe}} = r_{\max} - \Delta r$ is the manipulator maximum reach reduced by a margin Δr to account for uncertainties.

To guarantee (20), the vehicle must not only descend: it must also adjust its horizontal position so that the arm base stays within a safe distance from the nodule. Intuitively, while the landing task drives the vertical motion, an additional "distance-to-nodule" logic drives the horizontal motion (x-y) whenever the nodule is too far.

A simple first step is to compute the *linear position error* between a target point (e.g., the nodule frame or an associated grasp/scan target) and the current tool/base position:

$$\mathbf{e}_{lin} \;=\; {}^{w}\mathbf{p}_g - {}^{w}\mathbf{p}_t \qquad \text{(in the world frame } \langle w \rangle\text{)}, \tag{21}$$

where ${}^{w}\mathbf{p}_g$ is the desired goal point (e.g., nodule position) and ${}^{w}\mathbf{p}_t$ is the current tool point (or alternatively the arm base point). To enforce *horizontal* repositioning only, we use the projected error:

$$\dot{\bar{\mathbf{x}}}_{xy} \;=\; \begin{bmatrix} e_{lin,x} \\ e_{lin,y} \\ 0 \end{bmatrix}. \tag{22}$$

This produces a vehicle command that moves in x and y toward the nodule while leaving the z component to the dedicated landing task.

A principled way to implement the above is to introduce an additional safety task $\mathcal{T}_{DIST}$ that becomes active only if the nodule risks being outside the arm workspace.

Distance metric. Use the base-to-nodule distance

$$d \;=\; \|\mathbf{p}_n - \mathbf{p}_b\|. \tag{23}$$

Activation. Define an activation that turns on smoothly when d becomes too large:

$$a_{DIST}(d) \;=\; \mathrm{IncBell}\big(d_{\mathrm{safe}},\, d_{\mathrm{far}};\, d\big), \tag{24}$$

where $d_{\mathrm{safe}} < d_{\mathrm{far}}$, and:

- if $d \le d_{\mathrm{safe}}$ then $a_{DIST}(d) \approx 0$ (task inactive),
- if $d \ge d_{\mathrm{far}}$ then $a_{DIST}(d) \approx 1$ (task fully active).

This is consistent with the same bell-shaped activation style used for other inequality tasks (e.g., minimum altitude).

Reference generation. Let $\mathbf{p}_{b,\perp} = [p_{b,x},\, p_{b,y},\, 0]^\top$ be the arm-base position projected on the horizontal plane, and let $\mathbf{p}^*_{b,\perp}$ be a target base position that ensures reachability with margin. A natural choice is to place the base at a desired stand-off distance d_{stand} from the nodule:

$$\mathbf{p}^*_{b,\perp} = \mathbf{p}_{n,\perp} - d_{\mathrm{stand}}\,\hat{\mathbf{d}}_\perp, \qquad \mathbf{p}_{n,\perp} = [p_{n,x}, p_{n,y}, 0]^\top, \tag{25}$$

where $\hat{\mathbf{d}}_\perp$ is the unit horizontal direction toward the nodule (same quantity used for alignment). Then the reference horizontal velocity is

$$\dot{\bar{\mathbf{x}}}_{DIST} = k_d\, a_{DIST}(d)\,\Big(\mathbf{p}^*_{b,\perp} - \mathbf{p}_{b,\perp}\Big), \tag{26}$$

with gain $k_d > 0$ and saturation to cap the commanded velocity.

Jacobian. If the task variable is the base horizontal position $\mathbf{x}_{DIST} = \mathbf{p}_{b,\perp}$, then

$$\dot{\mathbf{p}}_b = \dot{\mathbf{p}}_v + \dot{\mathbf{R}}_{wv}\mathbf{p}_{vb}. \tag{27}$$

In practice, a robust and simple implementation for the inequality task is to map the task directly to the vehicle linear velocity in world coordinates (or body coordinates with rotation mapping). The Jacobian thus selects the vehicle linear part of the generalized velocity (and optionally includes a small coupling term due to rotation), then projects to x-y:

$$\dot{\mathbf{x}}_{DIST} = \mathbf{J}_{DIST}\,\dot{\mathbf{y}}, \qquad \mathbf{J}_{DIST} \approx \begin{bmatrix}\mathbf{0}_{2\times 7} & \mathbf{R}_{wv}(1{:}2, 1{:}3) & \mathbf{0}_{2\times 3}\end{bmatrix}, \tag{28}$$

i.e., the task mainly acts on the vehicle linear motion, leaving the arm joints unchanged. (A more accurate model can incorporate the rotational coupling term, but this approximation is typically sufficient for horizontal repositioning.)

Priority placement. $\mathcal{T}_{DIST}$ is an inequality safety task. To guarantee reachability, it should be placed *above* the landing task during the landing action, so that if reachability is at risk, the controller prioritizes moving closer over continuing to descend.

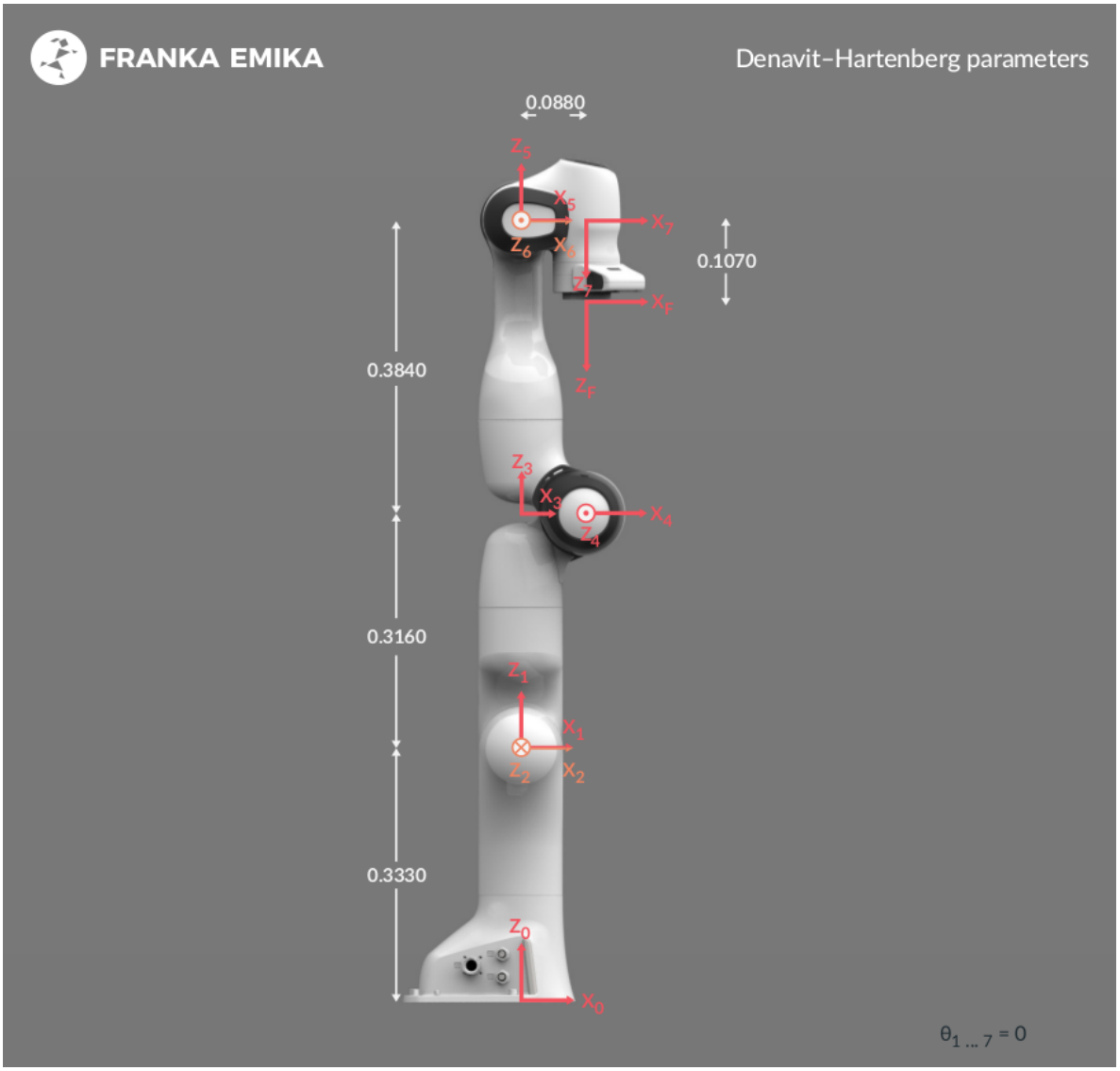


Figure 5: Robot Specifications

2 Exercise 2: Bimanual manipulation

In this exercise it is required to perform a bimanual manipulation by implementing the task priority algorithm considering two manipulators as a single robot. The manipulator adopted for this assignment is the Franka Panda by Emika (see Figure 5)). A simulation in python is provided, in order to visualize the two robots and test your Matlab implementation.

1. The transformations between the world and the base of each robot must be computed knowing that:
   (a) The left arm base coincides with the world frame.
   (b) The right arm base is rotated of π w.r.t. z-axis and is positioned 1.06 m along the x-axis and -0.01 m along the y-axis.
2. The transformation from each base to the respective end-effector is given in the code and is retrieved from the URDF model thanks to the *Robotic System Toolbox* of Matlab.
3. Then, define the tool frame for both manipulators; the tool frames must be placed at 21.24 cm along the z-axis of the end-effector frame and rotated with an angle of -44.9949 deg around the z-axis of the end-effector.

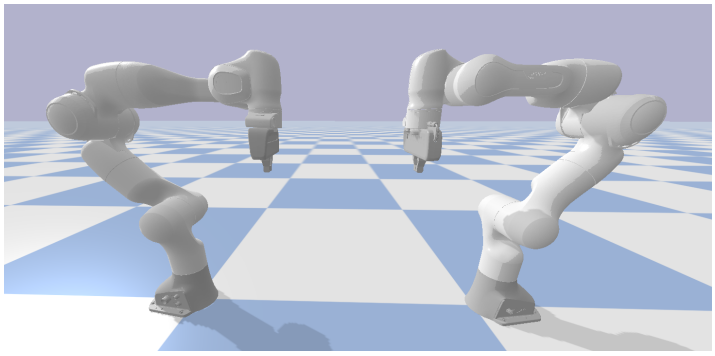


Figure 6: Initial position of the robots

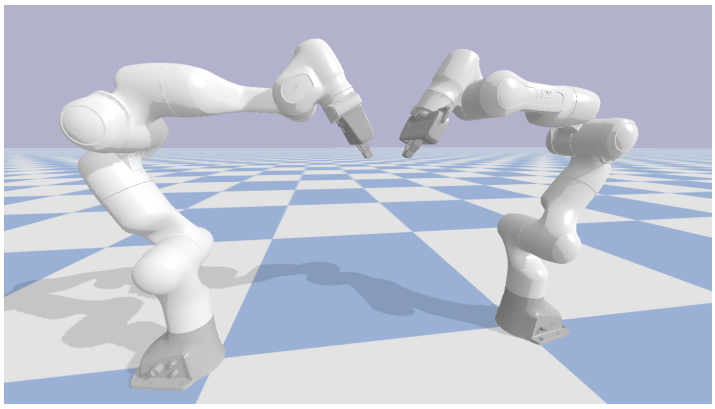


Figure 7: Relative orientation at the grasping position

4. Implement a "move to" action that includes the joint limits task.

5. The first phase foresees to move the tool frame of both manipulators to the grasping points, by implementing a "move to" action and its corresponding tasks. Define the goal frames for each manipulator such that their origin corresponds to the grasping points. **HINT**: the position of the grasping points can be computed by knowing the origin of the object frame ${}^{w}\mathrm{O}_o$ and the object length l_{obj}. The goal orientation of the tool frames is obtained by rotating the tool frames 30 deg around their y-axis; the manipulators should reach the configuration depicted in Figure 7.

$$ {}^{w}\mathrm{O}_o = [0.5, 0, 0.59]; \tag{29} $$

$$ l_{obj} = 10\,(\mathrm{cm}). \tag{30} $$

6. During this phase of the mission, the following safety tasks must be implemented: *end-effector minimum altitude* and *joint limits*. The joint limits specifications can be found in the datasheet of the robot. The minimum altitude from the table should be 15 cm.

Once the manipulators reach the grasping points, the second phase of the mission should start. Now, implement the Bimanual Rigid Grasping Constraint task to carry the object as a rigid body.

1. Define the object frame as a rigid body attached to the tool frame of each manipulator. **HINT:** Compute this quantity after reaching the grasping point.

2. Define the rigid grasp task.

3. Then, move the object to another position while both manipulators hold it firmly, e.g. ${}^{w}\mathrm{O}_g = [0.65, -0.35, 0.28]^\top (m)$

4. Note that the transition for the *Bimanual Rigid Constraint* should be a binary one, i.e., without smoothness. This is the nature of the constraint, i.e., either it exists or not. Modify the *ActionManager* class to take into account the different nature of this task (a constraint one).

Once the object has been moved to the required position you have to implement a final phase of the mission in which the joint velocities are set to zero, and every action is deactivated except for the minimum altitude task.

Repeat the simulation, using a goal position or by changing the grasping in such a way that one of the two manipulators activates multiple safety tasks, to stress the constraint task.

2.1 Q1: Report the unified hierarchy of tasks used and their priorities in each action.

State, control vector, and task-priority convention

In assignment 2 the two 7-DoF manipulators are treated as a *single* 14-DoF robot. The commanded generalized velocity is

$$\dot{\mathbf{y}} \;=\; \begin{bmatrix} \dot{\mathbf{q}}_L \\ \dot{\mathbf{q}}_R \end{bmatrix} \in \mathbb{R}^{14}, \qquad \dot{\mathbf{q}}_L, \dot{\mathbf{q}}_R \in \mathbb{R}^7, \tag{31}$$

and each task i is written in differential form as

$$\dot{\mathbf{x}}_i \;=\; \mathbf{J}_i(\mathbf{q})\,\dot{\mathbf{y}}, \qquad \dot{\mathbf{x}}_i \approx \dot{\bar{\mathbf{x}}}_i, \tag{32}$$

where $\dot{\bar{\mathbf{x}}}_i$ is the task reference (from a proportional law) and $\mathbf{J}_i$ is the Jacobian. A task activation matrix $\mathbf{A}_i$ (diagonal) smoothly enables/disables tasks, and the overall solver applies the tasks *in the order they appear in the stack* (highest priority first), projecting lower-priority objectives into the null space of higher-priority ones. The architecture does not support equal priority tasks, however obviously most priorities of left and right are the same on the real system.

Actions implemented

The mission is organized into three actions (phases):

1. **Action 1 — Move-to grasp points ("move to")**: drive both *tool* frames to the grasping frames while enforcing safety tasks (joint limits, minimum altitude).
2. **Action 2 — Bimanual rigid transport (constraint task)**: once grasp is achieved, enforce a rigid grasping constraint and move the *object* to the goal pose.
3. **Action 3 — Stop/Hold**: set velocities to zero by deactivating everything except minimum altitude.

Table 3: Unified hierarchy of tasks for Exercise 2

Task	Type	$\mathcal{A}_1$	$\mathcal{A}_2$	$\mathcal{A}_3$
Reach to G Point (Tool pose)	E	3		
Coop (Rigid grasping constraint and object motion)	E		4	
JL (Joint limits)	I	1	2	
AC (Min altitude, tool)	I	2	3	1
RBC (Rigid-body constraint)	E		1	

Unified hierarchy and priorities per action

The following tables report the task stacks and priorities *as used by the action stacks* (priority 1 = highest, applied first).

Action 1: Move-to grasp points

Prio	Task ID (concept)	Role
1	LJL (Joint limits, Left)	Prevent left joints from approaching limits (activation near limits)
2	RJL (Joint limits, Right)	Prevent right joints from approaching limits (activation near limits)
3	LAC (Min altitude, Left EE/tool)	Keep left tool/end-effector altitude above threshold (15 cm)
4	RAC (Min altitude, Right EE/tool)	Keep right tool/end-effector altitude above threshold (15 cm)
5	ReachL (Tool pose / position)	Move left tool frame to left grasping frame ${}^{w}T_{t,L} \rightarrow {}^{w}T^{*}_{t,L}$
6	ReachR (Tool pose / position)	Move right tool frame to right grasping frame ${}^{w}T_{t,R} \rightarrow {}^{w}T^{*}_{t,R}$

Action 2: Bimanual rigid transport (rigid grasp constraint)

Prio	Task ID (concept)	Role
1	RBC (Rigid-body constraint)	Enforces rigid grasp compatibility
2	LJL (Joint limits, Left)	Safety during transport (activate if limits approach)
3	RJL (Joint limits, Right)	Safety during transport (activate if limits approach)
4	Coop (Rigid grasping constraint / object motion)	Enforce rigid grasp kinematics and move object to ${}^{w}T_{o} \rightarrow {}^{w}T$

Action 3: Stop/Hold (minimum altitude only)

Prio	Task ID (concept)	Role
1	LAC (Min altitude, Left EE/tool)	Keep minimum altitude active as last safety layer
2	RAC (Min altitude, Right EE/tool)	Keep minimum altitude active as last safety layer

The transition activations between actions can be implemented over time, but we decided it would be a more adaptive approach if we transition over error-to-goal normed. Also, we implemented a **single action manager unit**, capable of handling cooperative (the distribution scheme in assignment 3) and non-cooperative tasks. In assignment 2 all tasks are non-cooperative. Associatedly, the carry task (`TaskCooperativeDistributed` is the same for assignment 2 and 3, with the iscoop property set to true or false). Further, the `generatePriorityList` function described in the previous section is also used for the manipulators.

The action definition stack

```
left_jointLimits_task = TaskJointLimits("L", "LJL", arm1.jlmax);
right_jointLimits_task = TaskJointLimits("R", "RJL", arm2.jlmax);
left_altitudeLimit_task = TaskAltitudeControl("L", "LAC", 0.15);
right_altitudeLimit_task = TaskAltitudeControl("R", "RAC", 0.15);
task_L_reach = TaskToolPosition("L", "ReachL", T_L_grasp);
task_R_reach = TaskToolPosition("R", "ReachR", T_R_grasp);
phase1_stack = {left_jointLimits_task, right_jointLimits_task, ...
               left_altitudeLimit_task, right_altitudeLimit_task, ...
               task_L_reach, task_R_reach};
phase1_ids = {'ReachL', 'ReachR', 'LJL', 'RJL', 'LAC', 'RAC'};
coop_task = TaskCooperationDistributed(wTog, arm1, arm2, l_T_o, r_T_o, false);
phase_coop_safety_stack = {left_jointLimits_task, right_jointLimits_task, ...
                          left_altitudeLimit_task, right_altitudeLimit_task, ...
                          coop_task};
phase_coop_safety_ids = {'LJL', 'RJL', 'LAC', 'RAC', 'Coop'};
phase3_stack = {left_altitudeLimit_task, right_altitudeLimit_task};
phase3_ids = {'LAC', 'RAC'};
```

```

actionManager = ActionManager_Comp();
actionManager.addAction(phase1_stack, phase1_ids);
actionManager.addAction(phase_coop_safety_stack, phase_coop_safety_ids);
actionManager.addAction(phase3_stack, phase3_ids);
```

2.2 Q2: What is the Jacobian relationship for the Joint Limits task? How was the task reference computed?

Task definition

The joint-limits task is a *joint-space* task. For one arm (left or right), define

$$\mathbf{x}_{JL} = \mathbf{q} \in \mathbb{R}^7, \qquad \dot{\mathbf{x}}_{JL} = \dot{\mathbf{q}}. \tag{33}$$

The objective is not to track a fixed posture, but to *push joints back* toward the safe region only when they approach their limits.

Jacobian relationship (14-DoF stacked system)

Because $\dot{\mathbf{x}}_{JL}$ is directly the joint velocity, the Jacobian is a selection matrix.

$$\dot{\mathbf{x}}_{JL,L} = \mathbf{J}_{JL,L}\dot{\mathbf{y}}, \qquad \mathbf{J}_{JL,L} = \begin{bmatrix}\mathbf{I}_7 & \mathbf{0}_{7\times 7}\end{bmatrix}. \tag{34}$$

$$\dot{\mathbf{x}}_{JL,R} = \mathbf{J}_{JL,R}\dot{\mathbf{y}}, \qquad \mathbf{J}_{JL,R} = \begin{bmatrix}\mathbf{0}_{7\times 7} & \mathbf{I}_7\end{bmatrix}. \tag{35}$$

In code:

```
if obj.ID=='L'
    obj.J = [eye(7), zeros(7,7)];
elseif obj.ID=='R'
    obj.J = [zeros(7,7), eye(7)];
end
```

Activation

Let $\mathbf{q}_{\max} \in \mathbb{R}^7$ be the joint-limit vector (symmetric model: $q_{\min,i} = -q_{\max,i}$), and define a safe bound

$$\mathbf{q}_{safe} = \alpha\, \mathbf{q}_{\max}, \qquad \alpha = 0.9. \tag{36}$$

A normalized proximity measure can be defined componentwise as

$$d_i = \frac{|q_i|}{q_{safe,i}}. \tag{37}$$

The activation is diagonal and increases when d_i approaches 1 (near the safe boundary) using an increasing bell-shaped function:

$$\mathbf{A}_{JL} = \mathrm{diag}(a_1, \ldots, a_7), \qquad a_i = \mathrm{IncBell}(0.9,\, 1.0;\, d_i), \tag{38}$$

so $a_i \approx 0$ deep in the safe region and $a_i \to 1$ near/at the limit.

Reference computation

The joint-limits task reference is designed to *push back* against the sign of the joint:

$$\dot{\bar{q}}_i = -k_{JL}\, \mathrm{sign}(q_i)\, e_i, \tag{39}$$

where $k_{JL} > 0$ is a gain and e_i is a nonnegative "limit violation" indicator that becomes nonzero only when the joint enters the unsafe band. A typical choice consistent with the activation definition is:

$$e_i = \max(d_i - 1,\, 0), \tag{40}$$

and then $\dot{\bar{q}}_i$ is saturated to respect actuator limits:

$$\dot{\bar{q}}_i \leftarrow \mathrm{sat}(\dot{\bar{q}}_i,\, \dot{q}_{max}). \tag{41}$$

2.3 Q3: What is the Jacobian relationship for the Bimanual Rigid Grasping Constraint task? How was the task reference computed?

Rigid grasp model and object frame definition

Once both tools reach the grasping poses, the object frame $\{o\}$ is modeled as a rigid body firmly attached to each tool frame $\{t_L\}$ and $\{t_R\}$. Let ${}^{w}T_{t,L}$ and ${}^{w}T_{t,R}$ be the current tool poses in world coordinates. At the grasp establishment instant, two constant tool-to-object transforms are computed:

$$ {}^{t_L}T_o \;=\; ({}^{w}T_{t,L})^{-1}\,{}^{w}T_o, \qquad {}^{t_R}T_o \;=\; ({}^{w}T_{t,R})^{-1}\,{}^{w}T_o. \tag{42} $$

During the transport phase, the object pose can be reconstructed independently from each arm as

$$ {}^{w}T_o^{(L)} = {}^{w}T_{t,L}\,{}^{t_L}T_o, \qquad {}^{w}T_o^{(R)} = {}^{w}T_{t,R}\,{}^{t_R}T_o, \tag{43} $$

where the relative tool-to-object transforms remain constant under the rigid grasp assumption.

Rigid-body constraint task formulation

A firm grasp implies that both arms must induce the *same* object twist, i.e.,

$$ {}^{w}\mathbf{v}_o^{(L)} \;=\; {}^{w}\mathbf{v}_o^{(R)}. \tag{44} $$

In the implementation, this is enforced through a **6D equality constraint task** defined as the difference between the two object twists:

$$ \mathbf{x}_{RBC} \;=\; {}^{w}\mathbf{v}_o^{(L)} - {}^{w}\mathbf{v}_o^{(R)} \;\in\; \mathbb{R}^6, \qquad \dot{\bar{\mathbf{x}}}_{RBC} \;=\; \mathbf{0}_6. \tag{45} $$

The 6D twist is expressed in world coordinates with the convention

$$ {}^{w}\mathbf{v}_o = \begin{bmatrix} \boldsymbol{\omega}_o \\ \mathbf{v}_o \end{bmatrix}, \tag{46} $$

where $\boldsymbol{\omega}_o$ and $\mathbf{v}_o$ are respectively the angular and linear velocity components.

Jacobian relationship

For a single arm, the tool twist is related to joint velocities by the geometric Jacobian

$$ {}^{w}\mathbf{v}_t = \mathbf{J}_t(\mathbf{q})\,\dot{\mathbf{q}}. \tag{47} $$

Let $\mathbf{r}$ be the vector from the tool origin to the object origin, expressed in world coordinates. By rigid-body kinematics, the twist at the object origin is obtained from the tool twist through the point-shift mapping:

$$ {}^{w}\mathbf{v}_o = \underbrace{\begin{bmatrix} \mathbf{I}_3 & \mathbf{0}_{3\times 3} \\ -[\mathbf{r}]_\times & \mathbf{I}_3 \end{bmatrix}}_{\mathbf{W}(\mathbf{r})} {}^{w}\mathbf{v}_t, \tag{48} $$

where $[\mathbf{r}]_\times \mathbf{a} = \mathbf{r} \times \mathbf{a}$. Therefore, an **object-point Jacobian** can be defined as

$$ \mathbf{J}_o(\mathbf{q}) = \mathbf{W}(\mathbf{r})\,\mathbf{J}_t(\mathbf{q}). \tag{49} $$

In the bimanual case, the two offsets are computed online as

$$ \mathbf{r}_L = \mathbf{p}_o^{(L)} - \mathbf{p}_t^{(L)}, \qquad \mathbf{r}_R = \mathbf{p}_o^{(R)} - \mathbf{p}_t^{(R)}, \tag{50} $$

where $\mathbf{p}_t$ and $\mathbf{p}_o$ are the tool and reconstructed object positions extracted from ${}^{w}T_t$ and ${}^{w}T_o$.

Stacked system constraint Jacobian

Let the stacked joint velocity vector be

$$\dot{\mathbf{y}} = \begin{bmatrix} \dot{\mathbf{q}}_L \\ \dot{\mathbf{q}}_R \end{bmatrix} \in \mathbb{R}^{14}. \tag{51}$$

Using ${}^{w}\mathbf{v}_o^{(L)} = \mathbf{J}_{o,L}\dot{\mathbf{q}}_L$ and ${}^{w}\mathbf{v}_o^{(R)} = \mathbf{J}_{o,R}\dot{\mathbf{q}}_R$, the rigid-body constraint task becomes

$$\mathbf{x}_{RBC} = {}^{w}\mathbf{v}_o^{(L)} - {}^{w}\mathbf{v}_o^{(R)} = \mathbf{J}_{o,L}\dot{\mathbf{q}}_L - \mathbf{J}_{o,R}\dot{\mathbf{q}}_R = \mathbf{J}_{RBC}\,\dot{\mathbf{y}}, \tag{52}$$

with the **6×14 constraint Jacobian**

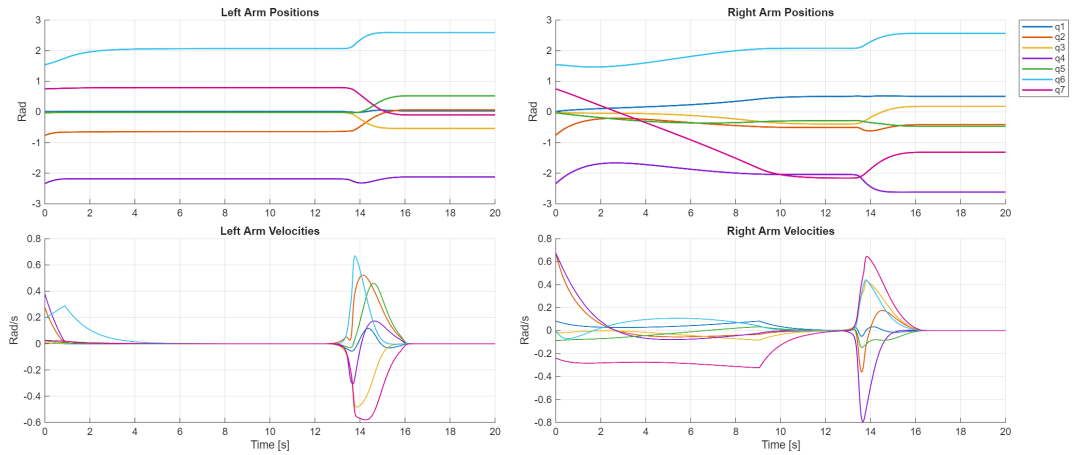
$$\mathbf{J}_{RBC} = \begin{bmatrix} \mathbf{J}_{o,L} & -\mathbf{J}_{o,R} \end{bmatrix} \in \mathbb{R}^{6\times 14}. \tag{53}$$

Since this is an equality constraint, the desired task reference is set to zero:

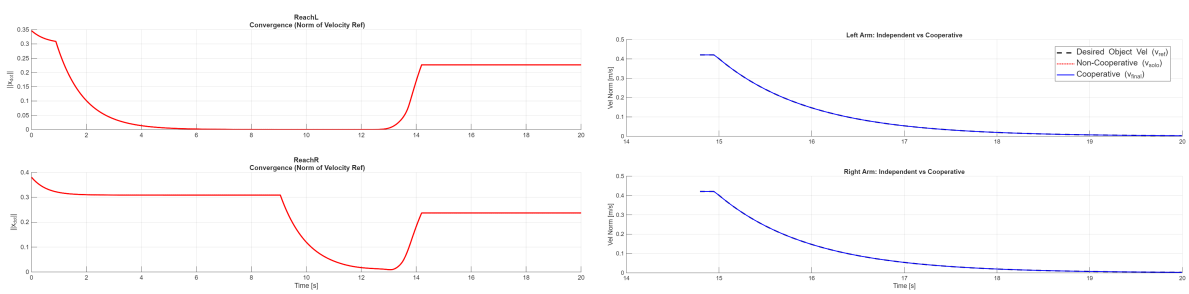
$$\dot{\bar{\mathbf{x}}}_{RBC} = \mathbf{0}_6, \tag{54}$$

so that the solver enforces consistency of the object twist reconstructed from the two manipulators. In practice, this task is placed at high priority during the cooperative transport phase to preserve the rigid grasp kinematics while other objectives (e.g., object motion tracking and safety constraints) are satisfied in the remaining null-space.

2.4 Q4: Comment the behaviour of the robots, using relevant plots. In particular, show the difference (if any) between the desired object velocity, and the velocities of the two end-effectors in the two cases. Add plots to show that the bimanual manipulation unfolds effectively (e.g., the distance between the tool frames is constant to show that the kinematic constraint is satisfied)



(a) Joint positions and joint velocities...



(a) Tool pose tracking error norm...

(b) Velocity tracking.

Figure 8: Joint-space evolution...

Phase 1 (move-to grasp points). Both tools move from the initial configuration to the grasping frames. The key expected signals are:

- decreasing Cartesian errors $\|\mathbf{e}_{t,L}\|$ and $\|\mathbf{e}_{t,R}\|$ (position/orientation),
- bounded joint velocities and smooth convergence to the grasp configuration,
- safety tasks mostly inactive (activations near zero) when the motion is feasible,
- safety tasks activating if we intentionally choose a stress configuration (e.g., very tilted grasp, difficult initial posture, or larger object length).

Phase 2 (rigid transport). Once the rigid grasp constraint is activated, both arms should move in a coordinated way to carry the object. Two core properties should hold:

1. **Object tracking:** the object pose approaches the goal ${}^{w}T_{o,g}$ and the object error decreases.
2. **Rigid grasp consistency:** the relative tool-to-tool distance remains (approximately) constant and the reconstructed object poses from left and right remain consistent.

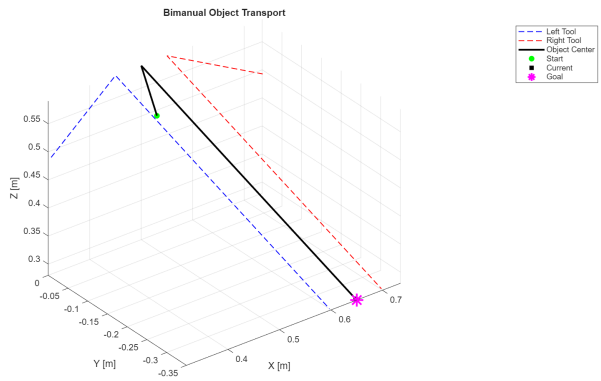


Figure 9

Phase 3 (stop/hold). Joint velocities are driven toward zero, and only minimum altitude remains active. Plots should show $\dot{\mathbf{q}}_L \to \mathbf{0}$ and $\dot{\mathbf{q}}_R \to \mathbf{0}$, while altitude is prevented from dropping below the threshold.

Motions of the arms

During Phase 2, the reference object twist is the 6D signal used by the constraint task. The implementation stores a global reference (often the average of left/right references),

$$ {}^{w}\mathbf{v}_{o,ref} \approx \frac{1}{2}\left({}^{w}\mathbf{v}_{o,ref}^{(L)} + {}^{w}\mathbf{v}_{o,ref}^{(R)}\right), \tag{55} $$

then this is the natural "desired object velocity" to plot.

Measured end-effector/tool velocities. From joint velocities, each tool twist can be computed as

$$ {}^{w}\mathbf{v}_{t,L} = \mathbf{J}_{t,L}\dot{\mathbf{q}}_L, \qquad {}^{w}\mathbf{v}_{t,R} = \mathbf{J}_{t,R}\dot{\mathbf{q}}_R. \tag{56} $$

Measured object velocities from each arm. Using the rigid offset $\mathbf{r}_L$ and $\mathbf{r}_R$, the object twist induced by each arm is

$$ {}^{w}\mathbf{v}_{o}^{(L)} = \mathbf{W}(\mathbf{r}_L)\,{}^{w}\mathbf{v}_{t,L}, \qquad {}^{w}\mathbf{v}_{o}^{(R)} = \mathbf{W}(\mathbf{r}_R)\,{}^{w}\mathbf{v}_{t,R}. \tag{57} $$

A good diagnostic plot is then the difference

$$ \Delta\mathbf{v}_o(t) = {}^{w}\mathbf{v}_{o}^{(L)}(t) - {}^{w}\mathbf{v}_{o}^{(R)}(t), \tag{58} $$

which should be close to zero during rigid transport. Also the position difference is a good measure for elastic pressures on the object, which has been plotted.

Rigid grasp validation: constant tool-to-tool distance

A simple geometric indicator that the constraint is satisfied is the distance between tool origins:

$$ d_{tt}(t) = \|\mathbf{p}_{t,L}(t) - \mathbf{p}_{t,R}(t)\|\,. \tag{59} $$

During Phase 2, $d_{tt}(t)$ should remain approximately constant. Although there are small oscillations, in the two robots' workspaces the oscillations never exceeded 0.5 cm.

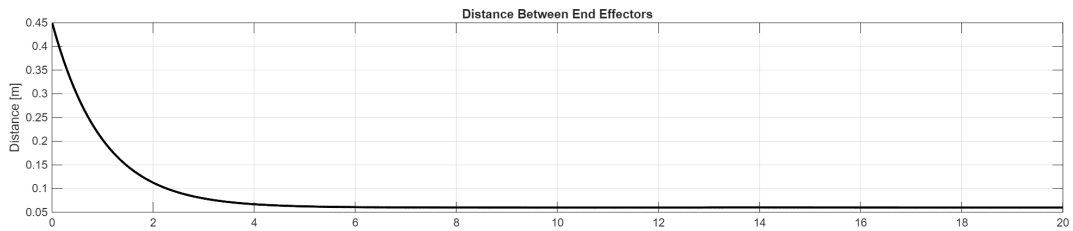
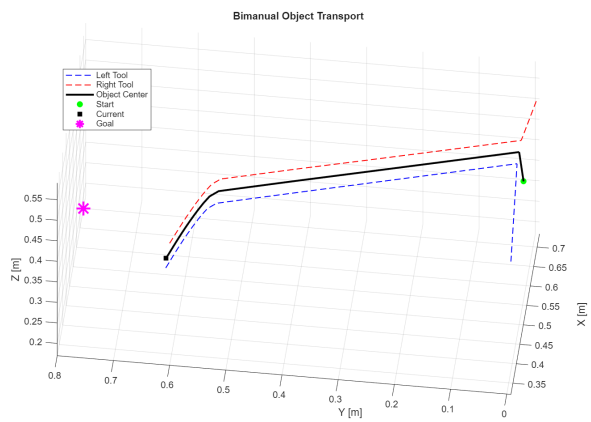
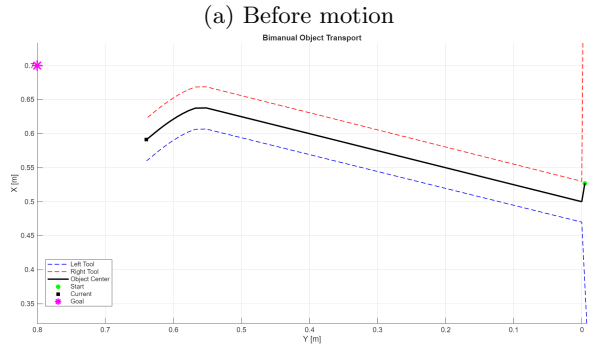


Figure 10: Distance between the two tool frames during rigid transport.

2.4.1 Stress testing



(a) Before motion



(b) Grasp configuration

For the stress test, we set a goal for [0.7, 0.8, 0.17], and the results show that the arms do not diverge even when they cannot reach the goal.

3 Exercise 3: Cooperative manipulation

In this exercise, it is required to perform cooperative manipulation by implementing the task priority algorithm considering the two Franka Panda manipulators as two distinct robots.

1. The first phase foresees to move the tool frames to the grasping points, by implementing the "move to" action for both manipulators. **Please note: each manipulator has his own Task Priority Inverse Kinematic Algorithm**. Define the goal frames for each manipulator such that their origin corresponds to the grasping points. **HINT**: the position of the grasping points can be computed by knowing the origin of the object frame ${}^{w}\mathrm{O}_o$ and the object length l_{obj}. The goal orientation of the tool frames is obtained by rotating the tool frames 20 deg around their y-axis.

$$ {}^{w}\mathrm{O}_o = [0.5, 0, 0.59]^\top (m); \tag{60} $$

$$ l_{obj} = 6\,(\mathrm{cm}). \tag{61} $$

   During this phase of the mission, the following safety tasks must be implemented: *end-effector minimum altitude* and *joint limits*. The joint limits specifications can be found in the datasheet of the robot. The minimum altitude from the table should be 15 cm.

2. Once the manipulators reach the grasping points the second phase of the mission should start. Implement the *Cooperative Rigid Constraint* task to carry the object as a rigid body.

   (a) Define the object frame as a rigid body attached to the tool frame of each manipulator.

   (b) Define the rigid grasp task.

   (c) You have to move the object to another position while both manipulators hold it firmly. The desired object goal position is

$$ {}^{w}\mathrm{O}_g = [0.60, 0.40, 0.48]^\top (m) \tag{62} $$

   (d) Compute the *non-cooperative* object frame velocities. **HINT:** Suppose manipulators communicate ideally and can exchange the respective end-effector velocities

   (e) Apply the coordination policy to the *non-cooperative* object frame velocities.

   (f) Compute the *cooperative* object frame velocities.

   Note that the transition for the *Cooperative Rigid Constraint* should be a binary one, i.e., without smoothness. This is the nature of the constraint, i.e., either it exists or not. Modify the *ActionManager* class to take into account the different nature of this task (a constraint one).

3. Once the object has been moved to the required position you have to implement a final phase of the mission in which the joint velocities are set to zero, and every action is deactivated except for the minimum altitude task.

Again, test it twice, once with the provided (reachable) goal, and then with a goal or with a different grasp configuration that triggers the activation of multiple joint limits or a joint limit and minimum altitude by one manipulator. The idea is that this second position should trigger the cooperation policy (i.e., the cooperative velocity of one manipulator should be different than the original desired object velocity, and that the cooperative velocity is very similar to its non cooperative one, showing that the other robot adapts).

3.1 Q1: Report the unified hierarchy of tasks used and their priorities in each action. Report cleary which action is used by TPIK1 and TPIK2 in the cooperative algorithm.

Similar to before In assignment 3 the two Panda arms are considered *two distinct robots*:

$$ \dot{\mathbf{q}}_L \in \mathbb{R}^7, \qquad \dot{\mathbf{q}}_R \in \mathbb{R}^7. \tag{63} $$

Mission actions (global phases)

The cooperative mission is divided into three actions (global phases):

1. **Action 1 — Independent move-to grasp points**: both arms move to their grasp frames. (Each arm executes a local "move to" action.)
2. **Action 2 — Cooperative rigid transport**: the object is carried as a rigid body to a goal pose, using a cooperative constraint/policy. (Each arm executes a local "cooperative transport" action.)
3. **Action 3 — Stop/Hold**: velocities set to zero, all actions deactivated except minimum altitude.

Cooperative setting: Distributed Coordination Policy

In Exercise 3, the two Panda arms are treated as *two distinct agents* that must physically collaborate to transport a common object. Unlike a centralized approach where one controller solves for $\dot{\mathbf{q}} \in \mathbb{R}^{14}$ simultaneously, here we employ a Distributed Coordination Policy.

The algorithm proceeds in four sequential steps at every control cycle:

1. **Independent Estimation (Solo TPIK):** Each agent ($i \in \{L, R\}$) first computes the object velocity $\dot{x}_{t,i}$ it *would* apply if acting alone, considering its own local safety constraints (e.g., joint limits).
2. **Consensus (Negotiation):** The agents exchange these proposed velocities. A weighted average $\hat{\dot{x}}_t$ is computed:
$$\hat{\dot{x}}_t = \frac{\mu_L \dot{x}_{t,L} + \mu_R \dot{x}_{t,R}}{\mu_L + \mu_R} \tag{64}$$
The weights μ_i are dynamic and proportional to the tracking error (difficulty). This creates an *emergent leader-follower behavior*: if one robot is close to a singularity or joint limit (high error), its weight increases, forcing the other robot to slow down or comply with the struggling agent's limitations.
3. **Kinematic Consistency (Projection):** The averaged velocity $\hat{\dot{x}}_t$ might violate the rigid grasp constraint (generating internal forces). It is projected onto the feasible subspace using the constraint matrix $C = H_L - H_R$:
$$\tilde{\dot{x}}_{coop} = (I - C^{\#}C)\hat{\dot{x}}_t \tag{65}$$
This ensures the resulting velocity $\tilde{\dot{x}}_{coop}$ induces pure motion without tearing the object.
4. **Coordinated Execution:** Finally, both agents execute a second TPIK optimization where tracking this negotiated $\tilde{\dot{x}}_{coop}$ is the highest priority task.

Mission Actions and Hierarchies

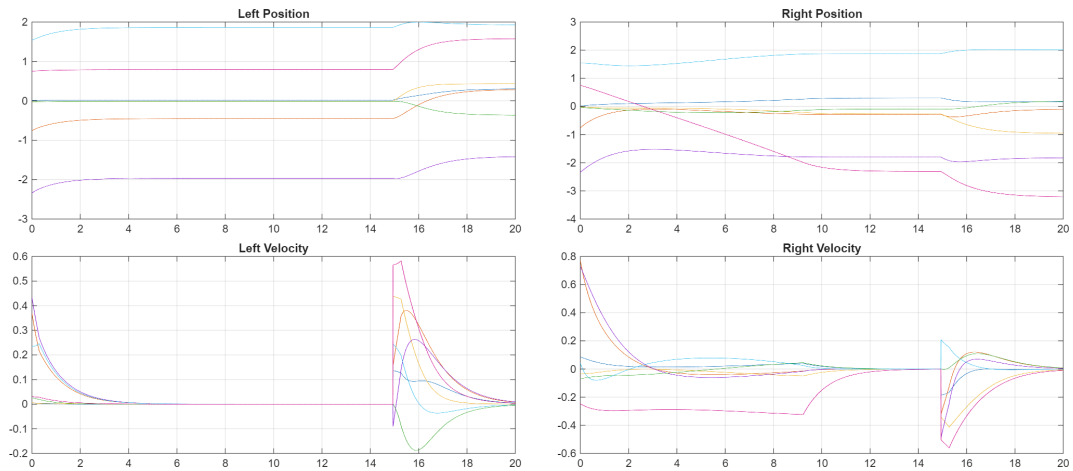
The mission is divided into three global actions. The specific hierarchies used by the local controllers (TPIK) for each action are reported below. Note that priority 1 is the highest. The priority hierarchies are the same as in assignment 2. However, here

- Action 1: Reach seperates the computation, so two separate solo TPIK will be run for each arm respectively. They perform no cooperation.
- Action 2: Carry uses the distributed double TPIK method for coordination between the two agents.
- Action 3: Stop will be solo TPIK, so separate computation.

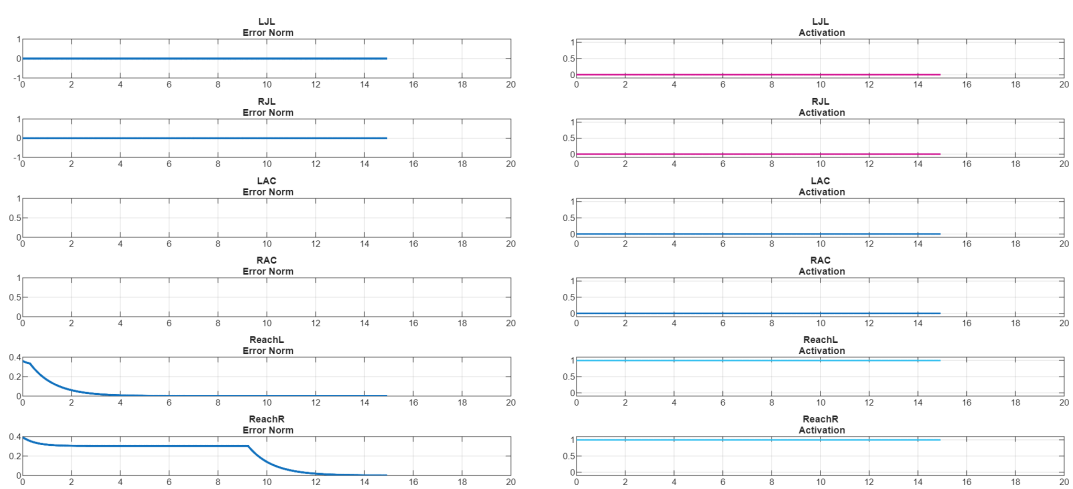
Note, there is only one action manager that separates the function based on the fact if the task has the iscoop property to true or not (`TaskCooperationDistributed(wTog, arm1, arm2, lTo, rTo, true)`). The cooperative rigid constraint is binary (exists / does not exist). Therefore, the switch into Action 2 is implemented as an almost instantaneous activation (no smooth blending). This ensures that once the grasp is established, the cooperative constraint is enforced immediately. This is vital for the final motion not to damage the object.

3.2 Q2: Comment the behaviour of the robots, using relevant plots. In particular, make sure there are differences between the desired object velocity and the non-cooperative Cartesian velocities at least in one simulation. Show also how the cooperative velocities of the two end-effectors behave. Add plots to show that the cooperation unfolds effectively (e.g., the distance between the tool frames is constant to show that the kinematic constraint is satisfied)

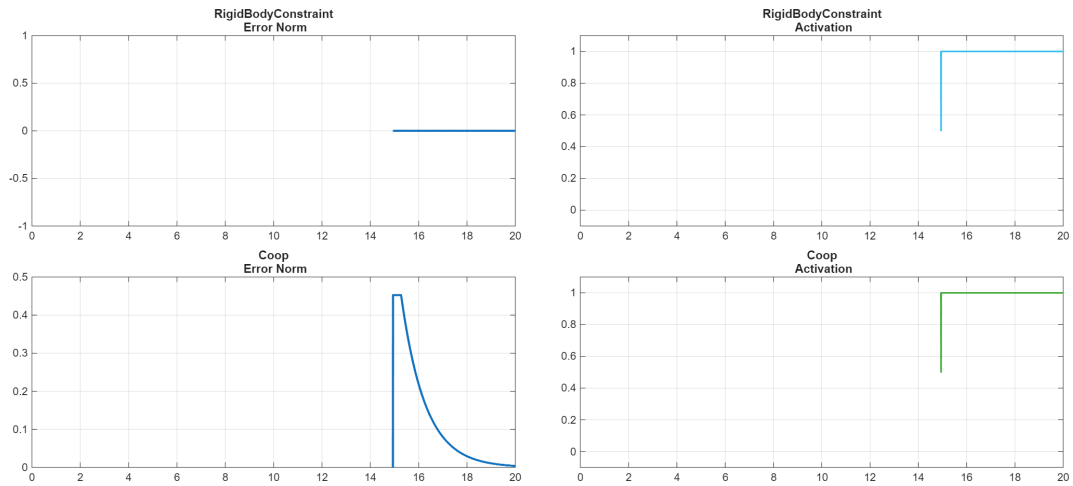
First three figures summarizes the joint-space evolution and the activation timeline of the main task stack. The joint position/velocity plots show that both manipulators generate smooth trajectories with bounded velocities. Most importantly, the transition between actions is designed to be near-continuous in velocity, meaning that the commanded Cartesian references do not introduce discontinuities when the action switches.



Joint positions and velocities. Smooth joint motions are obtained in both arms during the approach and the cooperative carry. Velocity peaks remain bounded and no impulsive behaviour is observed at the phase transitions.



Action 1 task activation timeline. The activation signal indicates whether each task contributes to the control stack. During approach, the reaching objectives (ReachL, ReachR) dominate the behaviour, while joint-limit and altitude tasks remain active to preserve feasibility.



Action 2 task activation timeline. During cooperative transport the cooperative motion task becomes active, and the rigid grasp compatibility is preserved while safety tasks remain enabled in the stack.

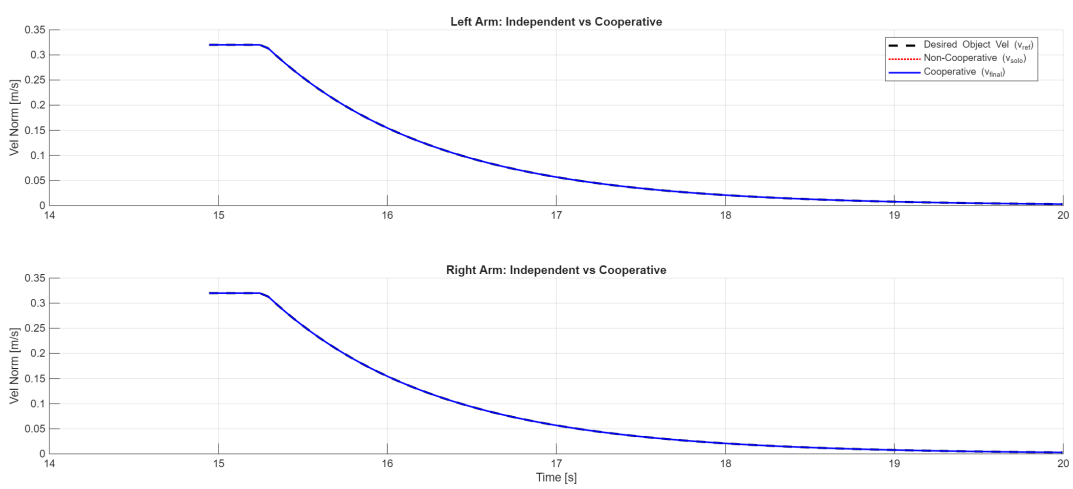
Desired vs non-cooperative vs cooperative Cartesian velocities

A key point of distributed cooperation is that the desired object velocity $\dot{\bar{\mathbf{x}}}_t$ (generated from the goal pose error) is not always directly achievable by both agents due to their kinematic limitations and the rigid grasp coupling. In this case, each arm would produce a different non-cooperative feasible Cartesian velocity (one per agent), denoted as $\dot{\mathbf{x}}_{t,L}$ and $\dot{\mathbf{x}}_{t,R}$. These non-cooperative velocities can differ from $\dot{\bar{\mathbf{x}}}_t$ and also from each other, particularly when one arm approaches a less dexterous configuration.

In the implemented coordination policy, the two non-cooperative velocities are blended and projected to generate a cooperative velocity $\dot{\tilde{\mathbf{x}}}_t$ that is feasible for the pair and consistent with the grasp constraint. As a result, in Action 2 both end-effectors track the same cooperative reference:

$$\dot{\mathbf{x}}_{t,L} \approx \dot{\tilde{\mathbf{x}}}_t, \qquad \dot{\mathbf{x}}_{t,R} \approx \dot{\tilde{\mathbf{x}}}_t.$$

The cooperative command is shared, while the individual non-cooperative commands reveal the mismatch that cooperation must resolve.



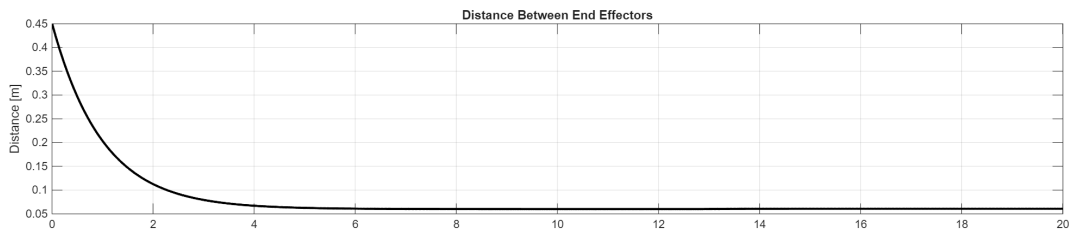
Cartesian velocity comparison. Desired object velocity $\dot{\bar{\mathbf{x}}}_t$ (reference generator), non-cooperative Cartesian velocities $\dot{\mathbf{x}}_{t,L}$ and $\dot{\mathbf{x}}_{t,R}$, and final cooperative velocity $\dot{\tilde{\mathbf{x}}}_t$.

Cooperation effectiveness and constraint satisfaction

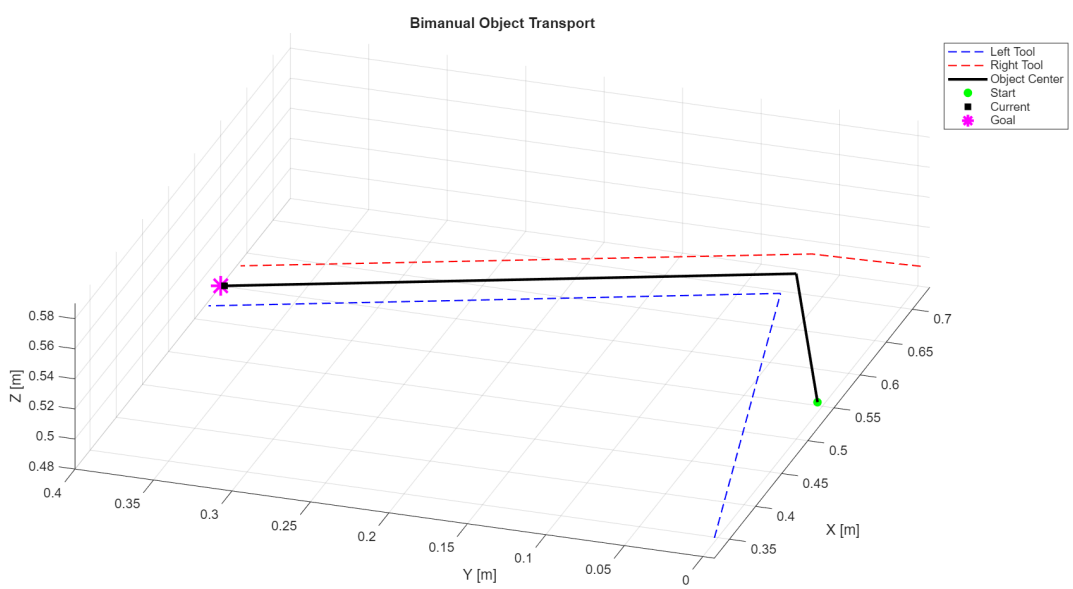
Under a rigid grasp assumption, the relative configuration between the two tool frames must remain constant during transport. A simple and effective indicator is the Euclidean distance between the tool origins,

$$d(t) = \|\mathbf{p}_{t,L}(t) - \mathbf{p}_{t,R}(t)\| \,.$$

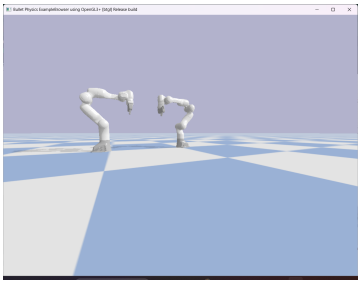
If cooperation is effective and the kinematic constraint is respected, this distance remains approximately constant after the grasp is established.
The following figure confirms the expected behaviour: after the approach phase, the distance stabilizes and stays constant throughout Action 2.



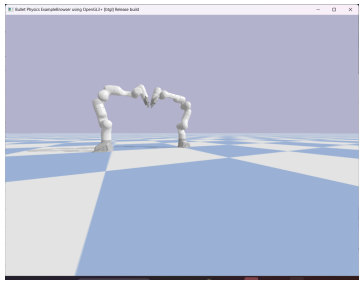
Distance between tool frames. After grasp establishment, the inter-tool distance remains constant during the cooperative carry, indicating satisfaction of the rigid grasp compatibility.



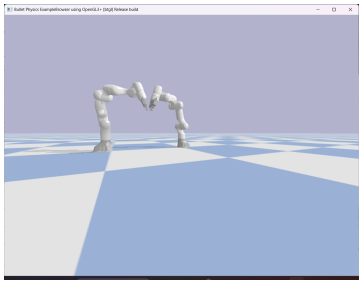
Tool trajectories and transported object path. The two end-effectors follow coordinated 3D trajectories. The object trajectory can be interpreted as the midpoint between the two tools, showing a smooth displacement toward the goal while maintaining relative grasp geometry.



(a) Before motion



(b) Grasp configuration



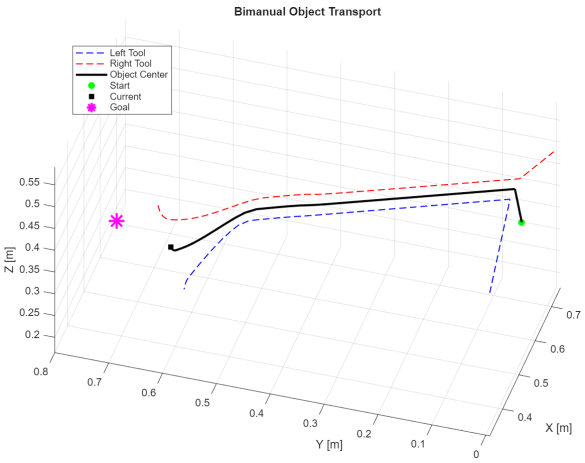
(c) After cooperative carry

Simulation. The sequence highlights (i) the initial configuration, (ii) the established grasp, and (iii) the final carried configuration after cooperative transport.

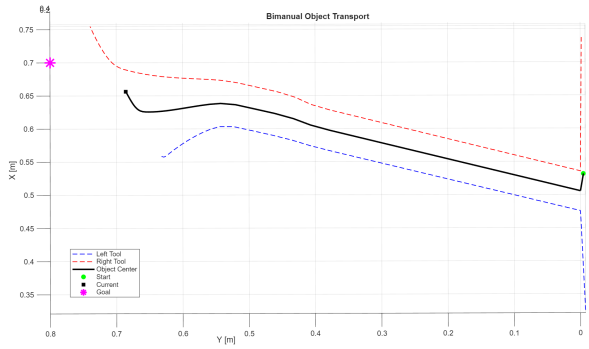
3.2.1 Stress Test

To further evaluate the method, a stress test is performed by setting a goal pose that is not equally reachable/dexterous for both arms (goal position [0.7, 0.8, 0.17]). In this scenario, the desired object velocity $\dot{\bar{\mathbf{x}}}_t$ may not be simultaneously feasible for both agents due to kinematic limitations, and cooperation becomes essential.

The resulting trajectory plots shows that the two arms still attempt to transport the object coherently, but the final convergence is degraded: the system cannot fully realize the intended goal pose while satisfying all constraints (kinematics, safety, and grasp compatibility). This is reflected by a less regular terminal behaviour and a slower reduction of the transport objective, which is the expected outcome in an infeasible or near-singular cooperative configuration.



3D cooperative trajectory (stress test)



Alternative view / end behaviour (stress test)

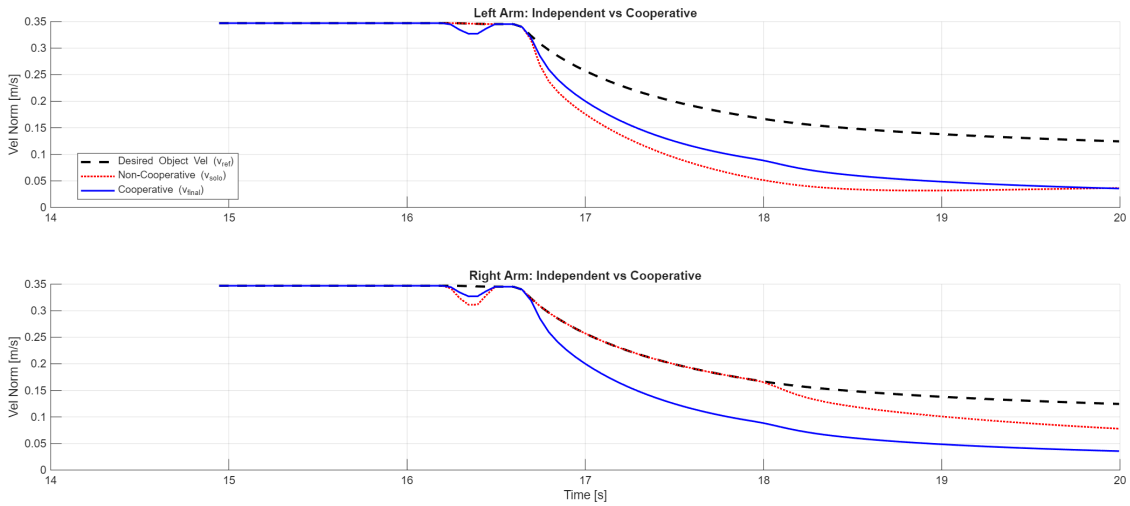
Stress test trajectory. A difficult goal highlights the difference between the desired object motion and what the two arms can jointly realize under rigid grasp and safety constraints.

Desired vs non-cooperative vs cooperative velocity norms in stressed case

Now we compares three velocity norms for each arm during the cooperative transport phase in a stressed case:

- **Desired object velocity** $\|\dot{\bar{\mathbf{x}}}_t\|$ (black dashed): velocity generated by the reference generator from the object goal error.
- **Non-cooperative feasible velocity** $\|\dot{\mathbf{x}}_{t,i}\|$ (red dotted): the Cartesian velocity that agent i would apply if acting alone (output of the first solve TPIK1 with action A_m).
- **Cooperative velocity** $\|\dot{\bar{\mathbf{x}}}_t\|$ (blue solid): the negotiated, projected and feasible cooperative velocity used as the shared reference in TPIK2 (action A_c), and tracked by both end-effectors.

The figure shows that once the cooperation phase starts, the non-cooperative velocities deviate from the desired reference. This indicates that the desired object motion cannot be directly realized by each arm independently due to kinematic feasibility limits. The coordination policy resolves this mismatch by generating a cooperative velocity that remains feasible for the pair and respects the rigid-grasp coupling: the cooperative profile lies between the desired and the non-cooperative profiles and decays smoothly as the goal is approached.



Velocity norm comparison for both arms. Black dashed: desired object velocity $\|\dot{\tilde{\mathbf{x}}}_t\|$. Red dotted: non-cooperative feasible velocity $\|\dot{\mathbf{x}}_{t,i}\|$ from solo TPIK1 (A_m). Blue solid: final cooperative velocity $\|\dot{\tilde{\mathbf{x}}}_t\|$ used as shared reference in TPIK2 (A_c).